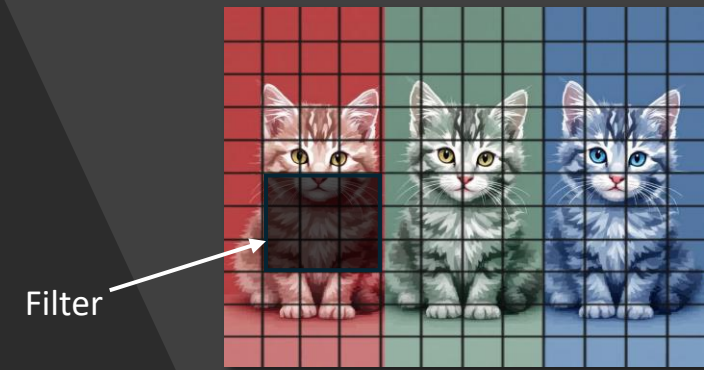


CNNs – Fusion Opportunity



Naive Execution Problems:

R: Read from Memory

W: Write to Memory

Input $\rightarrow$ R Conv2D $W \rightarrow$ R BatchNorm $W \rightarrow$ R ReLU $W \rightarrow$ Output

Each operation writes intermediate results to memory

- Separate kernel launches for each operation
- Poor data locality
- Underutilized hardware resources

Optimization Opportunity:

Input $\rightarrow$ R [Conv2D + BatchNorm + ReLU] $W \rightarrow$ Output

- Fused operations eliminate intermediate memory writes and reduce kernel launches.
- It is like taking a direct highway instead of stopping at every traffic light!

Kernel Fusion Techniques

Merging Operations for Efficiency

Common Fusion Patterns:

1. Conv-BatchNorm-Activation

- $conv_output = conv2d(input, weights) + bias$
- $norm_output = batch_norm(conv_output, scale, offset)$
- $final_output = relu(norm_output)$
- Each step writes to memory and reads from memory

After fusion, the above operations become a single (fused) kernel with no intermediate memory writes.

2. Element-wise Operation Fusion

Unfused:

Fused

- $result[i] = relu((x[i] + y[i]) \times z[i])$

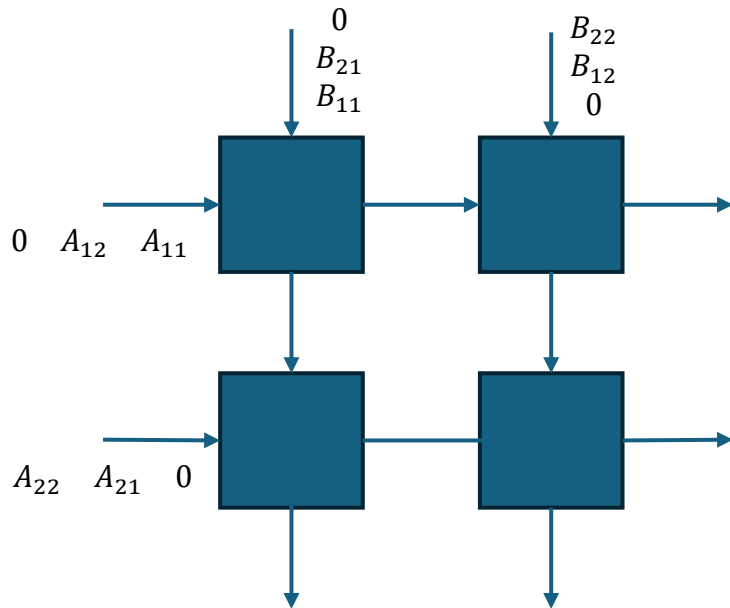
Memory Access Pattern Optimization

Optimizing Data Movement

- **Tiling for Systolic Arrays**
- **Data Layout Transformations**
- **Cache-aware Scheduling**

Tiling for Systolic Arrays

- A **systolic array** is a parallel computing hardware structure of connected processing elements (cells) that rhythmically process and transmit data
- TPUs** use systolic arrays as their core computational component to efficiently perform the high-throughput matrix multiplication required for AI workloads



A "tile" is a pre-defined, fixed-size chunk of a larger matrix.

$$A \times B = C$$

A_{11}	A_{12}	A_{13}	0
A_{21}	A_{22}	A_{23}	0
A_{31}	A_{32}	A_{33}	0
0	0	0	0

B_{11}	B_{12}	B_{13}	0
B_{21}	B_{22}	B_{23}	0
B_{31}	B_{32}	B_{33}	0
0	0	0	0

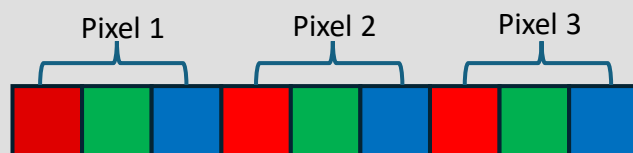
Both Matrices A and B are partitioned into fixed-size tiles that match the array's row bandwidth. Zeros pad the boundary, so every tile is the same size and enters on the same rhythm. As each A tile streams to the right and each B tile streams to the bottom, its values are reused across multiple columns of compute cells, letting one load of A and B participate in many multiplications before it exits.

Systolic 2x2 multiplication example: A tiles shift to the right and B tiles shift down. Each processing element multiplies its local A and B values and adds to a running sum. After several beats, the correct outputs emerge at the bottom/right without ever stopping the flow.

Data Layout Transformations

- N: Number of images in the batch.
- C: Number of Channels per image (e.g., 3 for Red, Green, Blue).
- H: Height of the image in pixels.
- W: Width of the image in pixels.

NHWC



- **Order:** Batch, Height, Width, Channels
- **What it means:** For each pixel, all its color/feature values are stored next to each other.
- **Typical:** TPUs/mobile NPUs; kernels that combine channels at a pixel
- Order in the above image: pixel 1(red)-> pixel 1 (green), pixel 1 (blue), pixel 2(red), pixel 2 (green), pixel 2 (blue), pixel 3(red), pixel 3 (green), pixel 3 (blue)

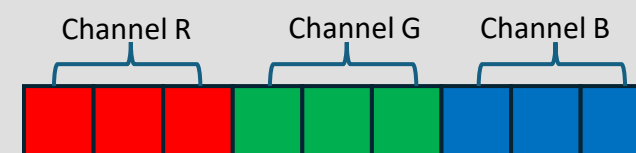
Reorder/transpose axes



Example permutation: [0, 3, 1, 2]
turns NHWC into NCHW

*No data changes—just the order
we store it.*

NCHW



- **Order:** Batch, Channels, Height, Width
- **What it means:** For each channel, all the pixels in the image are stored next to each other.
- **Good when:** You sweep over images one channel at a time.
- **Typical:** Convolution on many GPUs

For large matrix multiplies, we often go beyond NHWC/NCHW and use packed tile layouts that match the GEMM kernel

GEMM stands for General Matrix Multiplication. It's the mathematical name for the core operation that powers deep learning and scientific computing: multiplying two large matrices together.

Cache-aware Scheduling

Cache is a small, fast memory layer that sits between the CPU and main memory to reduce data access latency. It stores recently used (and nearby) data so the processor can reuse it quickly. L1 cache is the smallest and fastest, located closest to the CPU cores and split into instruction/data caches. L2 is larger but a bit slower, often private per core. L3 is much larger and slower than L1/L2, typically shared across cores on a chip, helping reduce trips to main memory by serving common data.

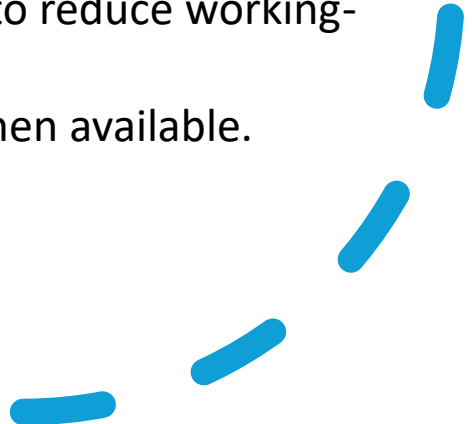
Why it matters:

- Minimizes cache misses by keeping hot data in faster caches (L1/L2) longer.

How it works:

- Reorder and group operations to reuse data while it's still in cache.
- Tile/block loops so working sets fit in L1/L2; promote shared data to L3-friendly batches.
- Align access patterns for spatial and temporal locality.

Practical tactics:

- Loop tiling/blocking, fusion, and interchange to reduce working-set size.
 - Prefetching and software-managed buffers when available.
- 

Memory Bandwidth Optimization

1. Memory Access Patterns

Core message

CPUs compute faster than they can fetch memory; prefetchers love sequential access.

Poor: outer loop over channels → memory jumps → cache misses → stalls.

Better: channels in the inner loop → adjacent addresses → effective prefetch → high utilization.

Impact

Can raise CPU utilization from ~20% to ~80% in image ops—about 4× faster.

Tip

Align loops with layout (e.g., NHWC vs NCHW). Keep the innermost loop traversing contiguous memory.

Memory Bandwidth Optimization

2. In-place Operations:

Core message

Avoid unnecessary copies; they double memory traffic and pressure.

Many ops (ReLU, clamp, simple norms, elementwise arithmetic) can overwrite inputs safely.

Benefits

Halves bandwidth for the op; reduces peak memory footprint.

Especially valuable on MCUs and also pays off on desktops by reducing DRAM traffic.

Safeguards

Only do in-place when no later step needs the original tensor.

Watch for aliasing with views/slices.

Memory Bandwidth Optimization

3. Memory Pool Management:

Core message

Dynamic allocation (malloc/new) adds overhead and fragmentation—bad for real time.

Preallocate a pool/arena and sub-allocate from it for tensors/workspaces.

Benefits

Predictable latency, fewer stalls, stable memory usage.

Matches how TF/PyTorch allocators and many embedded runtimes operate.

Embedded note

Only do in-place when no later step needs the original tensor.

Memory Bandwidth Optimization

Feeding the Compute Beast

The Bandwidth Challenge:

```
// Memory bandwidth calculation for inference
// Model: 100M parameters × 4 bytes/param = 400MB weights
// Inference rate: 100 inferences/second
// Required bandwidth: 400MB × 100 = 40 GB/s just for weights!
```

```
// Add activations and you easily exceed available bandwidth
```

Implementation Example:

```
// Tiled computation to maximize cache reuse
void optimized_conv2d() {
    for (int tile_y = 0; tile_y < height; tile_y += TILE_SIZE) {
        for (int tile_x = 0; tile_x < width; tile_x += TILE_SIZE) {
            // Process tile completely before moving to next
            // Maximizes reuse of loaded weights and inputs
            process_tile(tile_x, tile_y);
        }
    }
}
```

Optimization Strategies:

Weight Reuse

- Batch processing: Amortize weight reads across multiple inputs
- Temporal reuse: Keep weights in cache across operations

Activation Reuse

- Spatial reuse: Overlap in convolutional operations
- Channel reuse: Same spatial locations across feature maps

Precision Reduction

- 8-bit weights: 4x bandwidth reduction
- Mixed precision: Reduce precision where accuracy permits

Compression

- Sparse matrices: Store only non-zero values
- Huffman coding: Compress weight distributions